

NAME: A VELMURUGAN

EXPERIMENT NO: 1

QUESTIONS 10 / 10 completed

Single Linked List ▾

SINGLE LINKED LIST

"A college maintains a list of student roll numbers in the order they are registered. New students may join at different stages of the registration process. Develop a C program using a singly linked list to store the"

PROGRAM EDITOR

C ▾

Run Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 struct Node *head = NULL;
10
11 void display() {
12     struct Node *temp = head;
13
14     while (temp != NULL) {
15         printf("%d -> ", temp->data);
16         temp = temp->next;
17     }
18     printf("\n");
19 }
20
21 int main() {
22     insertBegin(10);
23     display();
24
25     insertEnd(20);
26     printf("After inserting 20 at end: ");
27     display();
28
29     insertPosition(15, 2);
30     printf("After inserting 15 at position 2: ");
31     display();
32
33     return 0;
34 }
```

INPUT

Your INPUT goes here!

```
76
77     insertEnd(20);
78     printf("After inserting 20 at end: ");
79     display();
80
81     insertPosition(15, 2);
82     printf("After inserting 15 at position 2: ");
83     display();
84
85     return 0;
86 }
87
```

OUTPUT

After inserting 10 at beginning: 10 -> NULL
After inserting 20 at end: 10 -> 20 -> NULL
After inserting 15 at position 2: 10 -> 15 -> 20 -> NULL

EXPERIMENT NO: 2

Deletion Operation ▾

DELETION OPERATIONS

"A company maintains employee records in a linked list. Employees may leave the organization, requiring their records to be removed. Write a C program to implement deletion operations from the beginning. ▾

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 struct Node *head = NULL;
10
11 void display() {
12     struct Node *temp = head;
13
14     while (temp != NULL) {
15         printf("%d -> ", temp->data);
16         temp = temp->next;
17     }
18     printf("NULL\n");
19 }
```

List: 10 -> 20 -> 30 -> 40
Delete node from beginning
Delete node from end
Delete node from position 2

```
93 deleteEnd();
94 printf("After deleting end: ");
95 display();
96
97 // Current list: 20 -> 30
98 deletePosition(2);
99 printf("After deleting position 2: ");
100 display();
101
102 return 0;
103 }
```

OUTPUT

```
Original List: 10 -> 20 -> 30 -> 40 -> NULL
After deleting beginning: 20 -> 30 -> 40 -> NULL
After deleting end: 20 -> 30 -> NULL
After deleting position 2: 20 -> NULL
```

EXPERIMENT NO:3

QUESTIONS 10 / 10 completed

Circular Linked List ▾

CIRCULAR LINKED LIST

"In a round-robin CPU scheduling system, processes are executed cyclically. Such applications can be represented using a circular linked list. Write a C program to create a circular linked list and display all

PROGRAM EDITOR

C ▾

Run Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *head, *n1, *n2, *n3, *n4;
11     struct Node *temp;
12
13     n1 = malloc(sizeof(struct Node));
14     n2 = malloc(sizeof(struct Node));
15     n3 = malloc(sizeof(struct Node));
16     n4 = malloc(sizeof(struct Node));
17
```

INPUT

10 -> 20 -> 30 -> 40

```
27
28     head = n1;
29     temp = head;
30
31     printf("Circular Linked List: ");
32
33     do {
34         printf("%d -> ", temp->data);
35         temp = temp->next;
36     } while (temp != head);
37
38     printf("HEAD");
39
40     return 0;
41 }
```

OUTPUT

Circular Linked List: 10 -> 20 -> 30 -> 40 -> HEAD

EXPERIMENT NO:4

Doubly Linked List ▾

DOUBLY LINKED LIST

"A music player maintains a playlist where users can move both forward and backward between songs. This can be efficiently implemented using a doubly linked list.

Write a C program to ▾

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *prev;
7     struct Node *next;
8 };
9
10 int main() {
11     struct Node *n1, *n2, *n3, *n4, *head, *temp;
12
13     n1 = malloc(sizeof(struct Node));
14     n2 = malloc(sizeof(struct Node));
15     n3 = malloc(sizeof(struct Node));
16     n4 = malloc(sizeof(struct Node));
17
18     n1->data = 10;
19     n2->data = 20;
```

10 -> 20 -> 30 -> 40

```
47     printf("NULL\n");
48
49     printf("Backward: ");
50     while (temp != NULL) {
51         printf("%d -> ", temp->data);
52         temp = temp->prev;
53     }
54
55     printf("NULL");
56
57     return 0;
58 }
```

OUTPUT

```
Forward: 10 -> 20 -> 30 -> 40 -> NULL
Backward: 40 -> 30 -> 20 -> 10 -> NULL
```

EXPERIMENT NO:5

Polynomials ▾

POLYNOMIALS

"In scientific computing applications, mathematical polynomials are often represented using linked lists where each node stores a coefficient and exponent. Write a C

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int coeff;
6     int exp;
7     struct Node *next;
8 };
9
10 struct Node* create(int coeff, int exp) {
11     struct Node *newNode = malloc(sizeof(struct Node));
12     newNode->coeff = coeff;
13     newNode->exp = exp;
14     newNode->next = NULL;
15     return newNode;
16 }
17
18 struct Node* add(struct Node *p, struct Node *q) {
19     struct Node *result = NULL, *temp = NULL, *newNode;
```

Polynomial 1: $5x^2 + 3x + 2$
Polynomial 2: $4x^2 + 6x + 1$

```
83     p->next->next = create(2, 0);
84
85     // Polynomial 2:  $4x^2 + 6x + 1$ 
86     q = create(4, 2);
87     q->next = create(6, 1);
88     q->next->next = create(1, 0);
89
90     result = add(p, q);
91
92     printf("Resultant Polynomial: ");
93     display(result);
94
95     return 0;
96 }
```

OUTPUT

Resultant Polynomial: $9x^2 + 9x^1 + 3x^0$

EXPERIMENT NO:6

Linked List ▾

LINKED LIST

"A library stores book identification numbers in a linked list. Develop a C program that searches for a given element and displays its position if found. Test Case 1"

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *head, *n1, *n2, *n3, *n4;
11     struct Node *temp;
12     int search, position = 1, found = 0;
13
14     n1 = malloc(sizeof(struct Node));
15     n2 = malloc(sizeof(struct Node));
16     n3 = malloc(sizeof(struct Node));
17     n4 = malloc(sizeof(struct Node));
18
19     n1->data = 10;
```

List: 10 -> 20 -> 30 -> 40
Search Element: 30

```
20     n2->data = 20;
21     n3->data = 30;
22     n4->data = 40;
23
24     head = n1;
25     n1->next = n2;
26     n2->next = n3;
27     n3->next = n4;
28     n4->next = NULL;
29
30     if (temp->data == search) {
31         printf("Element %d found at position %d", search, position);
32         found = 1;
33         break;
34     }
35
36     temp = temp->next;
37     position++;
38 }
39
40 if (found == 0)
41     printf("Element not found");
42
43 return 0;
44 }
```

OUTPUT

Element 30 found at position 3

EXPERIMENT NO:7

Singly Linked List-F ▾

SINGLY LINKED LIST-REVERSE

"In a logistics application, delivery locations are stored in a linked list. Sometimes the route needs to be traversed in reverse order. Write a C program to reverse a singly linked list and display the reversed

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *head, *n1, *n2, *n3, *n4;
11     struct Node *prev = NULL, *current, *next;
12
13     n1 = malloc(sizeof(struct Node));
14     n2 = malloc(sizeof(struct Node));
15     n3 = malloc(sizeof(struct Node));
16     n4 = malloc(sizeof(struct Node));
17
18     n1->data = 10;
19     n2->data = 20;
```

10 -> 20 -> 30 -> 40

```
25     n3->next = n4;
26     n4->next = NULL;
27
28     head = n1;
29
30     current = head;
31
32     while (current != NULL) {
33         next = current->next;
34         current->next = prev;
35         prev = current;
36         current = next;
37     }
```

OUTPUT

Reversed List: 40 -> 30 -> 20 -> 10 -> NULL

EXPERIMENT NO:8

Singly Linked List-1 ▾

SINGLY LINKED LIST-NODES

"A hospital maintains patient records using a linked list. To generate reports, the system needs to determine the total number of records currently stored. Write a C program to count and display the total

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *head, *temp;
11     int count = 0;
12
13     struct Node *n1 = malloc(sizeof(struct Node));
14     struct Node *n2 = malloc(sizeof(struct Node));
15     struct Node *n3 = malloc(sizeof(struct Node));
16     struct Node *n4 = malloc(sizeof(struct Node));
17     struct Node *n5 = malloc(sizeof(struct Node));
18
19     n1->data = 10;
```

10 -> 20 -> 30 -> 40
-> 50

```
28     n4->next = n5;
29     n5->next = NULL;
30
31     head = n1;
32     temp = head;
33
34     while (temp != NULL) {
35         count++;
36         temp = temp->next;
37     }
38
39     printf("Total number of nodes: %d", count);
40
41     return 0;
42 }
```

OUTPUT

Total number of nodes: 5

EXPERIMENT NO:9

QUESTIONS 10 / 10 completed

Single Sorted Linke ▾

SINGLE SORTED LINKED LIST

"Two departments maintain separate sorted lists of student IDs. For centralized processing, both lists need to be merged while preserving the sorted order. Write a C program to merge two sorted singly linked

PROGRAM EDITOR

C ▾

Run

Save

INPUT

List 1: 10 -> 30 -> 50
List 2: 20 -> 40 -> 60

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 struct Node* merge(struct Node *a, struct Node *b) {
10     struct Node *head = NULL, *temp = NULL, *newNode;
11
12     while (a != NULL && b != NULL) {
13         newNode = malloc(sizeof(struct Node));
14
15         if (a->data < b->data) {
16             newNode->data = a->data;
```

```
8
9 struct Node* merge(struct Node *a, struct Node *b) {
10     struct Node *head = NULL, *temp = NULL, *newNode;
11
12     while (a != NULL && b != NULL) {
13         newNode = malloc(sizeof(struct Node));
14
15         if (a->data < b->data) {
16             newNode->data = a->data;
17             a = a->next;
18         } else {
19             newNode->data = b->data;
20             b = b->next;
21     }
```

OUTPUT

Merged List: 10 -> 20 -> 30 -> 40 -> 50 -> 60 -> NULL

EXPERIMENT NO:10

Sorted Linked List- ▾

SORTED LINKED LIST-2

"A database contains sorted customer IDs, but due to data-entry errors, duplicate records have been created. Develop a C program that removes duplicate nodes from a sorted linked list and displays the updated

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *head, *temp, *nextNode;
11
12     int a[] = {10, 10, 20, 20, 30, 30};
13
14     head = NULL;
15     temp = NULL;
16
17     for (int i = 0; i < 6; i++) {
18         struct Node *newNode = malloc(sizeof(struct Node));
19         newNode->data = a[i];
20         newNode->next = NULL;
```

10 -> 10 -> 20 -> 20
-> 30 -> 30

```
41     }
42
43     printf("Updated List: ");
44
45     temp = head;
46     while (temp != NULL) {
47         printf("%d -> ", temp->data);
48         temp = temp->next;
49     }
50
51     printf("NULL");
52
53     return 0;
54 }
```

OUTPUT

Updated List: 10 -> 20 -> 30 -> NULL